

[DISCUSSION] Point-in-time Query && Change Data Capture for IVM

背景

增量物化视图 (IVM) 刷新时需要查询基表历史版本，以及版本之间的行级变更，用更常用的概念来描述这两种能力即 Point-in-time Query 和 Change Data Capture。

- Point-in-time Query (PITQ): 查询某个特定时间点或版本的数据
- Change Data Capture (CDC): 捕获两个版本之间的行级数据变更（包括插入、更新和删除）

目前常见的数据库或数据湖系统都具备这两种能力，比如 Snowflake、DataBricks、Iceberg、TiDB 等，主要的使用场景

- PITQ
 - Time Travel: 作为 TT 能力的一部分，TT 还会提供恢复能力，先查再恢复
 - IVM。。。
 - AI
- CDC
 - Streaming 计算: Flink/Spark Structured Streaming
 - IVM
 - 合规审计
 - scd type 2 表
 - 数据同步

对于同一种能力，不同场景有一些共同的需求，但也会存在差异。我们短期目标是覆盖 IVM，但长期可以拓展到更多场景，本文希望讨论下整体需求，有一个清晰的视图，然后针对 IVM 选择一个最小的实现落地，后面逐步演进。

本文档对需求、产品和技术实现做一个 high level 的讨论，最终更具体设计和实现后续可以再补充。这里把 PITQ 和 CDC 放在一起主要是为了 IVM 的连贯性，到具体实现部分可以再拆开。

目标

- 讨论不同场景对 PITQ 需求，IVM 的特点，以及技术路径
- 讨论不同场景对 CDC 需求，IVM 的特点，以及技术路径
- 讨论以 IVM 优先的最小落地路径

说明

有了后者可以实现前者，但支持前者不需要完整支持后者

- IVM version read < Point-in-time-query (Time Travel)
- IVM delta < CDC

Poin-in-time Query

需求分析

从其它系统看，Time Travel 对 PITQ 的需求是最全的，也就是说只要能满足 Time Travel，其它场景基本也能满足，Time Travel 是做什么的 (TODO，简单解释，参考[产品调研](#))，长期可以以覆盖 Time Travel 为目标，下面对比 Time Travel 和 IVM 的需求差异。

	IVM	Time Travel
目标	查询，主要关注数据	查询 + 业务逻辑恢复，关注元数据+数据，比如 schema、分区定义、分桶定义等等
SQL	DML + 影响数据的 DDL，比如 DROP/TRUNCATE/SWAP PARTITION等) partition 粒度的操作一般批量低频、数据量大，IVM 回退到全量刷新可能效率更高，有些用户也能接受，可以低优支持，比如 Applovin	DML + 影响表结构和数据的DDL，表结构的 DDL 比如 schema change、分区/分布策略、属性等
Schema	MV 刷新是为了和基表的最新状态保持一致，因此使用最新 schema 查询历史数据	语义上应该使用对应版本的历史 schema，实际各产品行为不一样 - Spark+Iceberg 使用历史 schema - Snowflake 使用当前 schema

用户接口	参考数据湖 IVM 实现，Analyze 阶段将版本信息注入到 plan 中，因此需要提供相应的 Java API 获取版本等信息	SQL API，通过 timestamp 或 version 指定要读取的版本，语法 代码块<pre>1 SELECT ... FROM t FOR TIMESTAMP AS OF '2026-02- 21 10:30:00'; 2 SELECT ... FROM t FOR VERSION AS OF 10;</pre>
保留时间	- 非固定，取决于 MV refresh interval，考虑到 IVM 用在实时场景，大部分小时粒度应该够用 - 可以 best effort，即便提前释放一些版本，MV 可以回退到全量刷新，主要是耗资源，不影响正确性	- 按固定时间，一般都是天粒度，比如 DataBricks 默认 7 天，Snowflake Standard Edition 默认 1 天 - 严格遵循，否则用户可能无法历史版本恢复，导致丢数据
保留版本	只需要保留 MV 上次 refresh 看到的最新版本 IVM 还需要保留上次 refresh 到当前版本的变更，这个和 PITQ 不完全一样	保留时间内所有 DML + DDL 产生的版本，要求能查询任意版本，保留成本可能比较高

总结下来相对于 IVM 对 PITQ 的需求是 Time Travel 的子集，主要表现为

- 覆盖核心的 DML 即可，其它 DDL 可以低优支持，遇到不支持的可以回退全量刷新
- 对元数据多版本能力要求低，只需要保留足够的元信息读取历史数据，schema 等元数据不需要
- 结合 IVM 实现提供 Java API 即可，不需要提供面向用户的 SQL API
- 可以使用更轻量的保留策略，降低历史版本维护成本

Table Version

Problem

无论是 PITQ 还是 CDC，都依赖“版本”的概念来描述表的状态：“我要读取表的哪个版本的数据”、“我要查询表的哪两个版本之间的变更”。StarRocks 缺少一个表粒度的“版本”定义和实现供用户使用，当前系统内部一些版本相关的定义不能满足这个需求

- Physical Partition Visible Version: 一个 physical partition 当前可读的版本，一个表可能由很多 physical partition 组成，每个 partition 维护自己的 version。表粒度的“版本”应该包括所有 partition 的 version 信息，而 physical partition version 不是表粒度的，无法满足需求

- Global Transaction Id (GTID): 标识一次成功提交的 DML 事务，在集群范围内提供单调递增的序列，但目前无法覆盖 DDL，比如 truncate partition、drop partition、schema change 等带来的数据变化。

因此我们首先需要对 "table version" 进行定义和实现。

定义和标识

Table State: 描述了一张表的逻辑结构和物理结构，包含了读写所需的必要信息。由 **Table Meta** (FE 管理并持久化的元数据) 和 **Table Data** (对象存储上的数据文件) 两层构成：

- **Table Meta** (FE 管理并持久化) :
 - **表定义:** 用户通过 DDL 声明的表逻辑结构，包括 Schema、分区、分布、索引、排序键、表属性等
 - **同步物化视图定义:** Rollup Index 的 MV 定义
 - **异步物化视图属性:** 异步 MV 特有的 Refresh 定义/状态、基表关联、Query Rewrite 配置等属性。这些属性仅对异步 MV 表适用，普通表无此组件
 - **数据分片:** 系统基于表定义自动创建的物理拓扑对象，包括 Physical Partition 实例和 Tablet 实例等
- **Table Data** (对象存储上的数据文件) : 实际存储的行数据及索引，包括 Rollup 数据

Table Version: 是 Table State 的版本，覆盖 Table Meta 与 Table Data 的变化。只要 Table State 的任一组件发生变化——无论是用户操作 (DDL、DML) 还是系统后台物理变化 (Compaction、range distribution tablet reshard、random 分桶 Physical Partition 动态新增等) ——都会产生一个新的 Version。

如何标识

- 系统内部使用 GTID，提供 Java API 根据 GTID 获取完整的 Table State，比如 IVM 通过 GTID 记录刷新进度
 - 对 GTID 进行扩展，所有改变 table state 的操作都分配一个 gtid
 - 与 table version 一一对应
- 面向用户使用 timestamp，系统内部将 timestamp 转成 GTID，GTID 本身包含了时间信息
 - "我要查询 t1 时刻数据", $\max\{gtid \mid \text{timestamp}(gtid) \leq t1\}$
 - "我要查询 t1 和 t2 之间的变更", $\Delta t1, t2 = \{gtid \mid t1 < \text{timestamp}(gtid) \leq t2\}$

技术挑战

对于 Time Travel 场景，Point-in-time Query 依赖的技术能力

1. 完整的 MVCC 能力，包括 Table Meta 和 Table Data，除了 DML，还能支持 DDL 修改 Table Meta 的需求，比如 schema change、partition evolution 等
2. 历史版本能够保留很长时间，比如天粒度
3. 查询可以基于指定版本的 Table State 执行

按照目前 StarRocks 机制，需要完善以下机制

1. Table Data 支持多版本，但 Table Meta 不支持

查询依赖 table meta，包括 schema、分区定义、分布定义、索引/rollup、数据分片 (partition/physical partition/tablet) 等。这些信息在 FE 数据结构 `OlapTable` 中维护，发生变化后直接原地更新，没有保留历史版本。如果某个 Table Version 后发生了 schema change、partition evolution、tablet reshard 等，查询历史版本依赖的 meta 就没有了，无法构造完整的 Table State。

比如

- 如果表有多个 partition，每次导入涉及部分 partition，并且每次的 partition 都不同，需要记录每次导入后的 partition 以及 visible version
- Fast schema change 加减列、修改列类型，需要保留历史版本的 schema 信息
- drop/truncate partition，旧 partition 暂存到 CatalogRecycleBin 等待回收，但是没有记录在哪个版本的 Table State 被使用，如果想构建之前版本的 state，需要记录之前版本的 partition name、range、id、physical partition、tablet id 等信息。
- 如果想访问 drop/truncate partition 前的版本，需要记录之前版本的 partition name、range、id、physical partition、tablet id 等信息。
- 涉及到重写数据的操作，比如非 fast schema change、修改分桶、修改 sort key、分区合并等，新 tablet 上不会保留历史版本数据，因此想要查询历史版本，必须保留历史 physical partition/tablet 元数据
- Range 分布 tablet reshard，目前新的 tablet 只保留最新版本的数据，想要访问历史版本，或者迁移到新 tablet 上，或者确保旧 tablet 不被清理，并且历史版本记录使用的旧 Tablet 版本

2. Dropped Tablet Vacuum 机制调整

StarRocks 目前通过一系列机制对 tablet dropped 保留一段时间，但是

- 这些 tablet 信息散落在各个地方 (CatalogRecycleBin、StarMgrMetaSyncer)，并且没有版本信息，无法拼凑出一个完整的可供查询使用的 Table State
- 不同场景下 tablet 保留机制不同，无法统一控制
 - Drop/Truncate partition，依赖 CatalogRecycleBin，保留时间 catalog_trash_expire_second。
 - Insert overwrite，依赖 CatalogRecycleBin，保留时间 partition_recycle_retention_period_secs
 - Range 分布 tablet reshared，依赖 StarMgrMetaSyncer，catalog tablet 信息 reshard 完成立刻清除，通过比对 catalog 和 staros shard 差异找到需要清理的 tablet，保留时间 star_mgr_meta_sync_interval_sec

如果想要支持 drop/truncate partition，CatalogRecycleBin 在判断 expire 需要考虑是否被 retention

如果想要支持 tablet reshard，*SplitTabletJob* 在 *cleaning* 时要考虑 retention，不能直接从 PhysicalPartition 中移除

3. FE 内存压力大

Time Travel 需要保留历史版本周期比较长，以天为粒度，并且需要保留所有相关 DML/DDI 的状态，元数据可能需要比较大的空间。当前 FE 元数据全部驻留内存，这种情况内存压力会比较大。

这些问题也不是这里独有的

- 之前的一些 feature 也会有各种元数据多版本的需求，但是通过一些局部多版本或 workaround 解决，没有一个统一的解决方案
 - Fast schema change v2 将历史版本 schema 保存在 Query Plan 和 SchemaChangeJob
 - Tablet reshard 只对用到的 materialized index 设计了多版本
- FE 内存压力一直是痛点，无需多言

我们需要一套彻底统一的方案来解决目前遇到的各种需求，并为未来的可扩展打下基础，而不是每个需求单独设计一些局部的方案来解决，越来越复杂，难以维护，容易出 bug，这个方案至少要包括如下两方面

1. 设计一套统一、完整的元数据多版本机制，包括数据结构、访问、retention 等，可以参考 TiDB
2. 历史版本元数据从内存迁移到冷存储来降低 FE 内存压力，包括 StarOs，比较可行的是使用内表作为冷存储

IVM 最小化实现

虽然说满足 Time Travel 就能满足 IVM，但如果按照 Time Travel 做技术方案设计，复杂度周期长，无法满足短期 IVM 的需求，因此按照 IVM 对需求进行裁剪，设计一个最小的可实现方案。

IVM 基于以下假设

- 仅支持 DML (INSERT / DELETE / UPDATE / 各类 LOAD),
 - 低优支持：drop tablet/partition、tablet reshard，数据结构上先可以支持，但 vacuum 后面支持
- 使用最新 schema，不需要保留历史 schema。
- 元数据只保留 MV 使用的版本，和 MV 数量有关，与当前 MV 模式相同
- partiton/tablet 被 drop/truncate/reshard 回退到全量刷，因此不需要修改 vacuum 机制

代码块

```
1  public class OlapTable {
2
3      // version of current table represented by GTID. Only updated after DML,
4      // can extend it to DDL or tablet shard in the future
5      @SerializedName(value = "versionId")
6      private long versionId = -1;
7  }
8
9  // minimal state for IVM, and only support DML excluding insert overwrite
10 public class TableState {
11
12     private final long tableId;
13     private final long versionId;
14
15     // partition state
16     // assume: no partition dropped/replaced and tablet reshard. PartitonInfo
    and tablet ids can be fetched from current OlapTable
17     private final Map<Long, LogicalPartitonState> partitionStates;
18 }
19
20 public class DeltaState {
21     private final long tableId;
22     private final Map<Long, LogicalPartitonDeltaState> partitionDeltaStates;
23 }
24
25 public class LogicalPartitonState {
26     private final long id;
27     private final String name;
28     private final Map<Long, PhysicalPartitionState>
29 }
```



```

30
31 public class LogicalPartitonDeltaState {
32     private final long id;
33     private final String name;
34     // only in new TableState
35     private final Map<Long, PhysicalPartitionState> newPartitions;
36     // only in old TableState
37     private final Map<Long, PhysicalPartitionState> droppedPartitions;
38     // in both new and old TableStates, but have different visible version
39     private final Map<Long, PhysicalPartitionDeltaState> droppedPartitions;
40 }
41
42 public class PhysicalPartitionState {
43     private final long id;
44     // To identify tablet reshard
45     private final long materializedIndexMetaId;
46     private final long visibleVersion;
47     private final long visibleTime;
48 }
49
50 public class PhysicalPartitionDeltaState {
51     private final long id;
52     // if reshard happens, the delta may include multiple-version materialized
53     // indexes. Each covers part of total deltas, for example, (1, 4], (4, 10], (10,
54     // 20]
55     private final List<MaterializedIndexDeltaState> indexDeltaStates;
56 }
57
58 public class MaterializedIndexDeltaState {
59     private final long metaIndexId;
60     private final long startVersion; // excluding
61     private final long endVersion; // including
62 }
63
64 public class TableVersionKeeper {
65     // Max time to keep for a table state after it's saved.
66     // After this duration, remove it no matter whether it's subscribed.
67     // This is a safe guard if someone forgets to unsubscribe it
68     private volatile long maxKeepTimeMs;
69
70     // version id -> table state
71     private final Map<Long, TableState> savedStates;
72     // subscribed states: version id -> count
73     private final Map<Long, AtomicLong> subscribedStates
74
75     // Subscribe the latest table state, and return the versoin id.

```



```

75 // 1. save the latest table state
76 // 2. add reference to avoid it be released
77 public Optional<Long> subscribeLatestTableState() {
78     // 1. construct TableState from the latestest OlapTable, and put it into
    savedStates
79     // 2. add reference, and update subscribedStates
80     // 3. return version id, or Optional.empty() if failed for some reasons
81 }
82
83 // Unsubscribe the version of table state. If no one subscribe it, it will
    be removed from the keeper
84 public void unsubscribeTableState(long versionId) {
85     // 1. verify the version exists
86     // 2. unsubscribe it, and update reference in subscribedStates
87     // 3. if the reference is 0, remove it from savedStates
88 }
89
90 public TableState getTableState(long versionId) {
91 }
92
93 public void removeExpiredTableState() {
94 }
95
96
97 // TODO For time travel in the future, need to save state for each DML and
    DDL.
98 // We can save the delta state for a new version to optimize the
    memory/storage cost
99 public saveTableState(long prevVersionId, long newVersionId, DeltaState
    state) {
100 }
101 }
102
103 public class TableStateUtils {
104
105     // Build the OlapTable used for query scan corresponding to the tableState.
106     // Limitations: Assumes Partitons/PhysicalPartitions still exist in the
    current OlapTable. Tablets in PhysicalPartition does not change
107     public static Optional<OlapTable> buildOlapTable(TableState tableState,
    OlapTable catalogTable) {
108         // 1. copy the current OlapTable for query from catalogTable
109         // 2. check whether Partitons/PhysicalPartitions still exists, and no
    reshard. If not match, return Optional.empty()
110         // 3. update copied OlapTable with tableState
111         // 3.1 remove partitons/physical partitons from OlapTable that do
    not in tableState
112         // 3.2 update PhysicalPartition visible version to tableState

```

```

113     }
114
115     // Compute delta state for change data capture
116     public static Optional<DeltaOlapTable> computeDeltaState(TableState
oldState, TableState newState, OlapTable catalogTable) {
117         // delta states include
118         // 1. dropped partitions: in oldState, not in newState
119         // 2. new partitions: in newState, not in oldState
120         // 3. changed partitions: both in old and new state, but with different
visible versions. Also need to consider multiple-version materialized index if
reshard happened
121     }
122 }
123
124 // IVM 如何使用
125 // 1. MaterializedView.AsyncRefreshContext 记录当前基表已经刷新的 version id, 当前
iceberg IVM 已经抽象好了数据结构 TvrTableSnapshot, 复用即可
126
127 public static class AsyncRefreshContext {
128     @SerializedName("baseTableInfoTvrVersionRangeMap")
129     private final Map<BaseTableInfo, TvrVersionRange>
baseTableInfoTvrDeltaMap = Maps.newConcurrentMap();
130
131     @SerializedName("tempBaseTableInfoTvrVersionRangeMap")
132     private final Map<BaseTableInfo, TvrVersionRange>
tempBaseTableInfoTvrDeltaMap = Maps.newConcurrentMap();
133 }
134
135 // 2. 刷新过程
136 long oldVersionId = 查找
MaterializedView.AsyncRefreshContext.baseTableInfoTvrDeltaMap
137 TableState oldState = TableVersionKeeper.getTableState(oldVersionId);
138
139 long newVersionId = TableVersionKeeper.subscribeLatestTableState().get()
140 TableState newState = TableVersionKeeper.getTableState(newVersionId);
141
142 OlapTable catalogTable = get it from current catalog
143 // scan old table
144 OlapTable oldTable = TableStateUtils.get(oldState, catalogTable);
145 // scan new table
146 OlapTable newTable = TableStateUtils.get(newState, catalogTable);
147 // scan delta (change data capture)
148 DeltaOlapTable deltaState = TableStateUtils.computeDeltaState(oldState,
newState, OlapTable);

```

这个数据结构可以支持 drop/truncate partitions、tablet reshard，但如果要完整支持，还要

- CatalogRecycleBin 不能把 TableVersionKeeper 引用的 partition 清理掉，从而查询时可以从 CatalogRecycleBin 找到
- Tablet Reshard 清理旧 tablet 时 不能把 TableVersionKeeper 引用的 materialized index meta 清理掉，从而查询时可以从 PhysicalPartition 找到

总结

短期

- P0
 - PITQ 只能查被 MV 引用的
 - 只支持 DML (除 insert overwrite)
- P1
 - 支持 drop/truncate partition、tablet reshard

长期

- 按照 time travel 场景
 - 支持任意 DML/DDl 版本查询 (元数据多版本数据结构设计)
 - 支持长时间保留 (冷存储)

Change Data Capture

CHANGES：DML 以及部分 DDL（比如 DROP PARTITION）产生的行级数据变更（包括插入、更新和删除），包括变更前后的数据，以及变更类型等元数据

Change Data Capture (CDC)：捕获 CHANGES 的动作

需求分析

CDC 主要有以下场景

- 增量物化视图：具体来说需要的是 delta，而不是逐条的 changes

- 流计算
- 安全审计
- 数据同步
- SCD (Slowly Changed Dimensions) Type 2

Row Tracking

存储层支持 Row Tracking，每一行数据包含如下信息

- ROW_ID (BIGINT): 逻辑行唯一标识，INSERT 生成全局唯一 row id，UPDATE 后 row id 保持不变
- ROW_VERSION (BIGINT): 数据的版本，INSERT 生成初始版本 (不一定从 0 开始)，UPDATE 后版本增加 (不一定连续)，row 之间的 version 没有关系，具体实现可以单独讨论，比如可以用 partition version 或 gtid 表示

基本概念

变更类型 (Change Type)

变更类型包括

- INSERT: 插入数据
- DELETE: 删除数据
- UPDATE_BEFORE: 更新操作生成两条变更，这个表示更新前的数据
- UPDATE_AFTER: 更新操作生成两条变更，这个表示更新后的数据

某些场景下 UPDATE 可以表示成 DELETE+INSERT，比如增量物化视图，见 3.4

- 前者可以保持语义的完整性，和 DELETE 区分开，在一些只需要 UPDATE_AFTER 的场景下可以跳过 UPDATE_BEFORE，比如同步到外部 kv 系统，只需要 overwrite。但需要将 BEFORE 和 AFTER 进行关联，因此生成成本要高
- 后者适用于不需要 UPDATE 完整语义的场景，比如增量 MV，优点是不需要将 BEFORE 和 AFTER 进行关联，生成变更的成本低，比如主键表只需要根据 old segment delete vector diff 以及 new segment 即可生成全部变更，不需要 old 和 new segment 之间 PK 进行映射。

CHANGES 组成

CHANGES = 数据列 + 元数据列

- 数据列：与表的列一致，可以只包含需要的列

元数据列

名称	类型	含义
CHNAGE_TYPE	TINYINT	变更类型: INSERT、DELETE、UPDATE_BEFORE、UPDATE_AFTER 备注: 如果配置了 UPDATE = INSERT + DELETE，可以去掉 UPDATE_BEFORE/UPDATE_AFTER
ROW_ID	BIGINT	逻辑行标识，同一行的所有变更具有相同的 row id
ROW_VERSION	BIGINT	产生变更的 row version，匹配的 UPDATE_BEFORE/UPDATE_AFTER 具有相同的 version

支持的表类型

表类型	支持的操作	变更类型
明细表	- DML: 只支持 append 类型，DELETE 成本高，暂不考虑 - DDL: TRUNCATE TABLE/PARTITION、DROP PARTITION	- INSERT - DELETE（DDL 产生）
主键表	- DML: INSERT INTO、INSERT OVERWRITE、DELETE、UPDATE，以及各类导入（STREAM LOAD / BROKER LOAD / ROUTINE LOAD 等） - DDL: TRUNCATE TABLE/PARTITION、DROP PARTITION	- INSERT - DELETE - UPDATE_BEFORE - UPDATE_AFTER
聚合表		

	- DML: 只支持 append，其它成本高 - DDL: TRUNCATE TABLE/PARTITION、DROP PARTITION	- INSERT（append 产生，aggregate 语义，对变更聚合后的结果，见下面示例） - DELETE（DDL 产生）
更新表	- 不支持	- 不支持

聚合表 CHANES 在 IVM 中的使用示例，Applovin 真实场景，基表 agg table，async mv 进行上卷

代码块

```
1 base agg table: key1, key2, key3, val1 sum
2 async mv: select key1, key2, sum(val1) group by 1, 2
3
4 // 导入数据: k=(1,1,1) 和 (2,2,2) 分别有两条
5 (1, 1, 1, 1) (1, 1, 1, 2) (2, 2, 2, 1) (3, 3, 3, 1), (2, 2, 2, 2)
6
7 // CHANGES: k=(1,1,1) 聚合后保留 1 条变更, k=(2,2,2) 保留 2 条原始变更
8 // 底层原理: 直接读取导入生成的增量 rowset 生成变更, rowset 存储的是 aggregate 后的结果
9 (1, 1, 1, 3) (2, 2, 2, 1) (3, 3, 3, 1), (2, 2, 2, 2)
```

消费粒度

批量模式，以版本为粒度，可以指定读取某个版本或连续几个版本的变更，但不能只读取某个版本的部分变更

接口

查询接口

- 指定 timestamp 或 version 查询，适合 Ad-hoc 查询，查询任意 changes

代码块

```
1 -- Option 1: Table Function
2 SELECT * FROM table_changes("tbl", ts1, ts2);
3 SELECT * FROM table_changes("tbl", v1, v2);
4
5 -- Option 2: dedicated CHANGES clause like Snowflake and Spark
```

```

6  -- Snowflake https://docs.snowflake.com/en/sql-reference/constructs/changes
7  -- Spark https://issues.apache.org/jira/browse/SPARK-55668
8  SELECT * FROM tbl CHANGES FROM VERSION v1 TO v2;
9  SELECT * FROM tbl CHANGES FROM TIMESTAMP v1 TO v2;

```

- 定义 **STREAM** 对象，自动管理 offset，只能查询 [last offset, current) 的 changes，类似 Snowflake 等系统

代码块

```

1
2  -- 初始 offset 1
3  CREATE STREAM stream ON tbl;
4
5  -- 查询 offset 到最新版本之间的 CHANGES, [offset 1, now]
6  SELECT * FROM stream;
7
8  -- DML 中查询 stream，执行成功后自动更新 offset 到最新位置
9  INSERT INTO target_tbl SELECT * FROM stream;
10
11
12  SELECT * FROM stream;

```

- 对接 Flink/Spark structured streaming，使用 SDK 通过 RPC 与 CN 进行数据传输
 - PULL 模式：类似当前 connector SCAN，客户端发一个特殊查询，指定要消费的范围，服务端执行一个特殊 plan，通过 RPC 返回给客户端
 - PUSH 模式：客户端发起订阅，服务端有新的 CHANGES 自动推送给客户端，实时性更高，暂不考虑

查询 Plan

OlapTableChangesScanNode -> IVM

顺序

这里的顺序指存储层 scan 返回的数据，即查询 `select * from tbl CHANGES`

- Row 之间无序

- 同一个 version+row 下的 update_before/update_after 之间无序，并且不保证相邻
- 一次消费多个 version 时，version 之间无序，比如先读取 version 1 的 changes，在读取 version 2 的 changes

如果需要保序，可以在计算层做，主要考虑

- 存储层首要任务是并行 scan，避免 IO 成为瓶颈，比如高频实时导入产生很多小文件，或大导入，version 之间、同一个 version 文件之间、同一个文件内部都可能并行 scan。存储层在并行 scan 的基础上支持保序，不一定比计算层做效率高以及可 scale，并且引入额外复杂度。有些场景在存储层做可能会效率高些，比如只有一个 version，并且数据量比较小，可以作为后续优化不通用
- 不同场景对顺序要求不同，在计算层做更灵活
 - IVM：本质是批处理，version 之间以及 update_before/update_after 之间都不需要有序。后面会提到 Net Changes，其实现会对同一个 row 下的 changes 排序进行合并，但这个相对比较宽松的
 - 流计算(Flink等)：以 CHANGE/record 为粒度处理，中间结果对下游可见，如果不是严格按照 version + update_before/update_after 顺序，下游可能看到非预期的结果。TODO: 举个例子

Update 语义

Update 产生的两条 changes 两种表示方式

- update_before + update_after，保留完整的 update 语义，流计算或者审计场景下有用，实现复杂，要将两条 change 进行关联
- Delete +insert，适合不需要 update 语义的场景，比如 IVM，实现上更轻量

两种都支持，可以配置，IVM 使用 delete+insert

Net Changes

将多个 version 同一个 row 的 changes 合并，比如 TODO 举例。

适合不需要知道每次变更过程的场景，比如 IVM，而审计这种场景不能 net changes。数据进入 IVM 前 net 好处

- 减少数据量，提高增量计算效率
- 简化 IVM 对 changes row_version 处理

TODO: 举 join 例子，左边有 changes，右表没有 changes，如果 ivm 前不 net，join 的数据量增加，merge 到 mv 时需要排序，IVM 增量算法需要考虑 row_version 如何映射到结果

```

1  join
2
3  A (id, name) B(id, val) (不变)
4
5  A changes: -(1, 1) (1, 2), -(1, 2) (1, 3) -> ivm join -> -(1, 1, 1), (1, 2,
1), -(1, 2, 1) (1, 3, 1) -> mv

```

可以通过 session 变量决定是否开启 net changes, Net Changes 在计算层完成, 主要考虑和顺序类似, 存储层专注于 scan IO 效率优化。

OlapTableChangesScanNode

Project/Filter PushDown

TODO

- Project: 支持数据列, 元数据列是否可以
- Filter 哪些列可以下推 row_id / row_version / change_type / data columns 哪些可以下推, 需要考虑是否影响 net changes 实现、update_before/update_after 匹配

技术挑战

基本流程

- 根据 FE 元数据 (table version) 找到变化的 physical partition/tablets, 以及 visible version range
- 每个 tablet 根据 visible version range 生成 changes

核心问题

- 核心: 对于明细表、聚合表以及只有 insert 的主键表, changes 可以从 delta rowset 顺序 scan 文件读取, 而主键表 delete/update 旧值读取, 列式存储+随机读取可能影响性能
 - 如何尽量避免随机
 - 导入性能 vs 查询性能, who is first class
- 其它优化
 - 高频实时或大导入下, 文件多, 数据量大时, 如何 scan 提升 IO 效率
 - Net changes 如何生成

整体方案

1. 导入时生成，主键表更新 primary index 能够拿到 delete/update_before 旧的 rssid，根据 rssid 读取旧值，新旧值配对写到 change log，查询时读取 change log
 - 优点
 - 利用 pk 已有机制配对 before 和 after，无需额外机制
 - 查询读取 change log 效率高
 - 缺点
 - 读取旧值影响导入性能
 - change log 增加存储和维护开销
 - 对于 net changes 场景不一定要保留每次导入的 changes，可以通过 compaction 后的数据比较来获取，对于小导入有优势
 - Change log 需要存储所有列，但查询可能只需要部分列和数据(project/filter)
2. 导入时增加一些轻量的元信息辅助定位 delete/update_before，从 segment 读取旧值移到查询时来做
 - 优点
 - 不影响导入，没有额外存储开销
 - 查询时如果涉及多个导入变更，可以把同一个 segment 的旧值查询攒批，提高效率
 - 运行时 filter / project
 - 缺点
 - 增加一些元数据开销
 - Scan 效率没有提前生成 change log 高

采用方案2

- 导入性能优先，尤其是实时导入场景，对抖动敏感，IVM 本身就是异步，相对要求低
- 引入 Change log 维护复杂度高

MVCC

需要确保时间窗口内的 table state 不能被清理，如果要查询任意 version 之间的 changes，需要保留任意 DML+DDL 信息，FE 元数据面临的挑战和 PITQ 相同。IVM 最小化实现，假设只保证 IVM 刷新时需

要的old和new之间的 changes 能访问，这样和 PITQ 就统一了，等以后 MVCC 支持任意版本自然就扩展了

元数据

1. 明细表和聚合表不变

2. 主键表

- 增加如下 bitmap vector
 - 旧 segment 当前只记录更新后的全量 delete vector，无法知道 delta，增加这次导入的 delete change type bitmap vector、update_before change type bitmap vector
 - 新 segment 增加 update_after change type bitmap vector
 - update_before/update_after bitmap vector 相当于是做了关联
- 查询时先找到 delta rowset，根据这些 bitmap vector 可以推导出每个 segment 要读取的数据以及 change type，segment 包括 segment 内部都可以并行 scan 提高 scan 效率

TODO: 结合代码说明元数据数据结构，举例说明导入时如何生成

并行 Scan 例子

以主键表为例说明 scan 过程，多个导入，覆盖 insert/update/delete，并且期间有 compaction，体现可以并行，比如，包括导入间的

加 delta bitmap vector 也可以解决 tablet reshard 后 changes 不在一个 tablet 上的情况，每个 version 的 changes 只依赖自己 version 的 tablet metadata，不需要跨 tablet 比对，举例说明

Net Changes

将同一个 `row_id` 下的多条 CDC 变更合并为**最小等价的 net changes**，使下游消费者应用 net changes 后得到的最终状态，与应用全部原始变更后的状态一致。

首末变更类型判定

变更类型：`0` = INSERT，`1` = DELETE，`2` = UPDATE_BEFORE，`3` = UPDATE_AFTER。配对的 UPDATE_BEFORE 和 UPDATE_AFTER 共享相同的 `row_version`。

对每个 `row_id`，根据 `row_version` 确定两个关键属性：

- **first_type**: 最小 `row_version` 处的变更类型，取该版本下 `MIN(change_type)`。
 - 0 (INSERT) : 该行在本批次内新建。
 - 1 (DELETE) : 该行在本批次内被删除（仅当只有单条变更时出现）。
 - 2 (UPDATE_BEFORE) : 该行在本批次之前已存在，首次变更为更新。
- **last_type**: 最大 `row_version` 处的变更类型，取该版本下 `MAX(change_type)`。
 - 0 (INSERT) : 该行在本批次内新建（仅当只有单条变更时出现）。
 - 1 (DELETE) : 该行在本批次结束时已被删除。
 - 3 (UPDATE_AFTER) : 该行在本批次结束时仍存在，持有更新后的值。

当只有单条变更时，`first_type` 和 `last_type` 相同，只可能为 INSERT (0) 或 DELETE (1)。

合并规则

规则 1：仅有单条变更

若某个 `row_id` 下只有一条变更记录（只可能是 INSERT 或 DELETE），原样输出，无需合并。

规则 2-5：多条变更合并

若某个 `row_id` 下有多条变更记录，根据 `first_type` 和 `last_type` 的组合进行合并：

#	first_type	last_type	输出	语义说明
2	INSERT (0)	UPDATE_AFTER (3)	1 条: <code>(row_id, max_ver, INSERT, max_after_val)</code>	新建后被更新，净效果等价于以最终值直接插入
3	INSERT (0)	DELETE (1)	0 条	新建后被删除，变更相互抵消
4	UPDATE_BEFORE (2)	UPDATE_AFTER (3)	2 条: <code>(row_id, max_ver,</code>	经历一次或多次更新，净效果等价于

			UPDATE_BEFORE, min_before_val) + (row_id, max_ver, UPDATE_AFTER, max_after_val)	从原始值直接更新 到最终值
5	UPDATE_BEFORE (2)	DELETE (1)	1 条: (row_id, max_ver, DELETE, min_before_val)	先被更新后被删 除, 净效果等价于 直接删除, val 携带 原始值

版本对齐: 规则 4 和 5 中, 输出记录的 `row_version` 统一使用 `max_ver`, 确保配对的 UPDATE_BEFORE / UPDATE_AFTER 具有相同版本, 符合标准 CDC 语义。

计算层可以使用窗口函数来做, local shuffle

代码块

```
1 ##### 示例
2
3 可以把 CHANGES 看做如下的明细表
4
5 ```sql
6 CREATE TABLE changes (
7     row_id INT,
8     row_version INT,
9     change_type INT,
10    val INT
11 ) DUPLICATE KEY(row_id)
12 DISTRIBUTED BY HASH(row_id) BUCKETS 5;
13 ```
14
15 | 列名 | 说明 |
16 |-----/-----/
17 | `row_id` | 逻辑行标识。INSERT 时分配, 后续 UPDATE 继承, DELETE 后不再复用。 |
18 | `row_version` | 产生该变更的版本号 (DML 序列号), 每个 DML 对应一个版本。 |
19 | `change_type` | 变更类型 |
20 | `val` | 变更携带的数据。 |
21
22 原始变更数据:
23
24 ```
```

row_id	row_version	change_type	val
1	1	0 (INS)	10
2	1	0 (INS)	10
2	3	2 (BEF)	10
2	3	3 (AFT)	20
3	1	0 (INS)	10
3	5	1 (DEL)	10
4	2	2 (BEF)	10
4	2	3 (AFT)	20
4	4	2 (BEF)	20
4	4	3 (AFT)	30
5	2	2 (BEF)	10
5	2	3 (AFT)	20
5	5	1 (DEL)	20
6	3	1 (DEL)	50

Net changes 输出:

row_id	row_version	change_type	val	命中规则
1	1	0 (INS)	10	#1 单条 INSERT
2	3	0 (INS)	20	#2 <INS, AFT> → 以最终值插入 (row_id=3 命中规则 #3, 抵消无输出)
4	4	2 (BEF)	10	#4 <BEF, AFT> → 从原始值更新到最终值
4	4	3 (AFT)	30	#4
5	5	1 (DEL)	10	#5 <BEF, DEL> → 携带原始值删除
6	3	1 (DEL)	50	#1 单条 DELETE

SQL 实现

利用表按 `row\_id` 分桶的特性避免全局 shuffle。

使用两层 CTE, 通过窗口函数在每个 `row\_id` 分区内完成分类和过滤。

```

sql
WITH base AS (
    SELECT
        *,
        MIN(row_version) OVER (PARTITION BY row_id) AS min_ver,
        MAX(row_version) OVER (PARTITION BY row_id) AS max_ver,
        COUNT(*) OVER (PARTITION BY row_id) AS cnt
    FROM changes

```

```

71 ),
72 classified AS (
73     SELECT
74         *,
75         MIN(CASE WHEN row_version = min_ver THEN change_type END)
76             OVER (PARTITION BY row_id) AS first_type,
77         MAX(CASE WHEN row_version = max_ver THEN change_type END)
78             OVER (PARTITION BY row_id) AS last_type
79     FROM base
80 )
81 SELECT
82     row_id,
83     CASE WHEN cnt > 1 THEN max_ver ELSE row_version END AS row_version,
84     CASE
85         WHEN first_type = 0 AND last_type = 3 THEN 0
86         WHEN first_type = 2 AND last_type = 1 THEN 1
87         ELSE change_type
88     END AS change_type,
89     val
90 FROM classified
91 WHERE
92     -- 规则 1: 单条变更, 原样输出
93     cnt = 1
94     -- 规则 2: <INSERT, UPDATE_AFTER> → 输出 max_ver 的 after, type 改为 INSERT
95     OR (first_type = 0 AND last_type = 3
96         AND row_version = max_ver AND change_type = 3)
97     -- 规则 3: <INSERT, DELETE> → 不输出
98     -- 规则 4: <UPDATE_BEFORE, UPDATE_AFTER> → 输出 min_ver 的 before + max_ver
99     -- 的 after
100     OR (first_type = 2 AND last_type = 3
101         AND row_version = min_ver AND change_type = 2)
102     OR (first_type = 2 AND last_type = 3
103         AND row_version = max_ver AND change_type = 3)
104     -- 规则 5: <UPDATE_BEFORE, DELETE> → 输出 min_ver 的 before, type 改为 DELETE
105     OR (first_type = 2 AND last_type = 1
106         AND row_version = min_ver AND change_type = 2)
107 ;
108 ```
109 **特点**: 无需 JOIN, 全部在窗口函数内完成。所有 `PARTITION BY row_id` 与分桶键一致, 可本地执行。
110

```


在计算层做，还有一个考虑就是 tablet reshard，reshard 前后同一个 row 的 changes 来自不同 tablet，无法在存储层做，在计算层做比较合适，注意此时**不能用 local shuffle**，考虑到 reshard 频率低，可以接受。TODO 举例说明

小文件问题

场景：高频实时导入，mv refresh 间隔大，比如小时，会有很多小文件

假设：只需要 net changes，update = delete+insert，IVM 场景适用

思路：比较 old 和 new 两个版本的文件，new 是经过 compaction 的，小文件数量少，提高 scan 效率

特例：聚合表，compaction 后 aggregate，无法获取变更

问题：解决 carry-over row，在计算层通过 row_version deduplication

统计信息

增量数据统计，删除了多少，新增了多少，文件数等

- 动态查询，从 tablet metadata 收集 (推荐先做这个)
- 导入结果附带这些信息，在 FE 缓存，可以先不持久化

总结

- 短期
 - P0
 - CHANGES 语句只能查被 MV 引用的 changes，即 [old version, 当前最新)，因为 FE 元数据没有记录中间的 state，无法比对出变化的 partition 和 visible version
 - 只支持 DML (除 insert overwrite)
 - Row tracking
 - Net changes
 -
 - P1
 - 支持 drop/truncate partition、tablet reshard
 - 小文件 compaction 优化

总结

- PITQ + CDC 整体架构
- 整体 roadmap